# Comparison Between Newton's Method and Steffensen's Method

Prepared By:
**Sultan Yasir Alasami**
**Rayan Abdullah AlShahri**
**Osama Khaled Bayounis**

Prepared For:
**Dr. Ahmed Harbawi**

# Table of Contents


| | |
|---|---|
| Introduction to Newton's Method and its Formula | 03 |
| Potential Issues with Newton's Method | 04 |
| Horizontal Tangent Issue | 04 |
| Initial Guess Far from the Root | 05 |
| Cycling Behavior | 05 |
| MATLAB Implementation of Newton's Method | 06 |
| MATLAB Script Input (Newton's Method) 1/2 | 06 |
| MATLAB Script Input (Newton's Method) 2/2 | 07 |
| MATLAB Script Output (Newton's Method) | 07 |
| Introduction to Steffensen's Method and it's Formula | 08 |
| Potential Issues with Steffensen's Method | 09 |
| Sensitivity to the Initial Guess | 09 |
| Susceptibility to Cycles | 09 |
| Double Function Evaluation Overhead | 09 |
| MATLAB Implementation of Steffensen's Method | 10 |
| MATLAB Script Input (Steffensen's Method) 1/2 | 10 |
| MATLAB Script Input (Steffensen's Method) 2/2 | 11 |
| MATLAB Script Output (Steffensen's Method) | 12 |
| Conclusion | 13 |

# Introduction to Newton's Method and its Formula

Newton's Method is a highly effective numerical technique utilized to approximate the roots of equations. This approach relies on constructing linear approximations through tangent lines drawn on the function's curve. To begin this iterative procedure, an initial guess, typically denoted by x0, must be provided, and it is recommended to select this guess near the actual root for optimal accuracy.

The iterative formula used to calculate subsequent approximations depends directly on the previous estimation. Provided the function f(x) is continuous, and its first derivative f′(x) is known and not equal to zero at the initial approximation, the method is mathematically defined by:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad n = 0, 1, 2, 3, \ldots$$

In this expression, x(n+1) denotes the next estimated root based on the current approximation x(n).

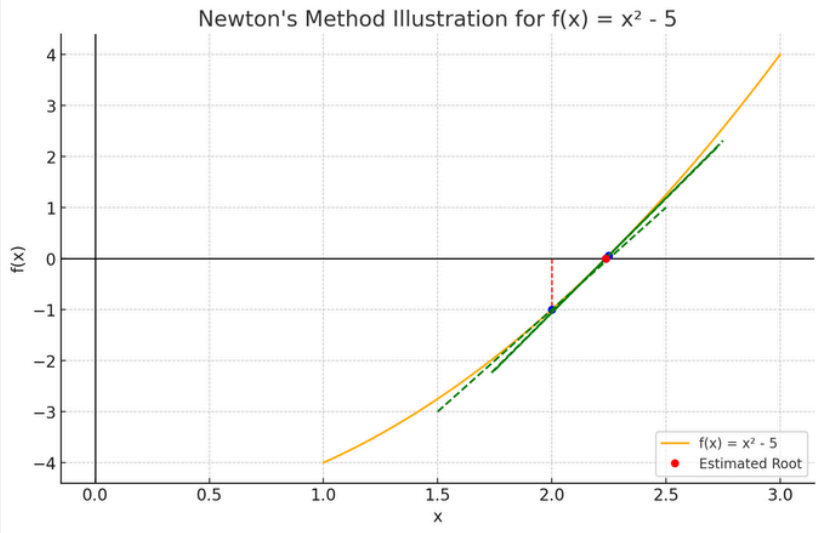
## *Illustrative Example:*

Consider the function f(x)=x^2-5. Using the initial approximation x(0) = 2,, Newton's method iterations proceed as follows:

| Iteration (n) | x(n) | f(x(n)) | f'(x(n)) | x(n+1) |
|---|---|---|---|---|
| 0 | 2 | -1 | 4 | 2.25 |
| 1 | 2.25 | 0.0625 | 4.5 | 2.236111111 |
| 2 | 2.236111111 | 0.000192902 | 4.472222222 | 2.236067978 |
| 3 | 2.236067978 | 3.71689×10−93.716 | 4.472135955 | 2.236067978 |

The iterations are halted when subsequent estimates converge and show no significant difference up to the desired accuracy level (here, five decimal places).

# Introduction to Newton's Method and its Formula



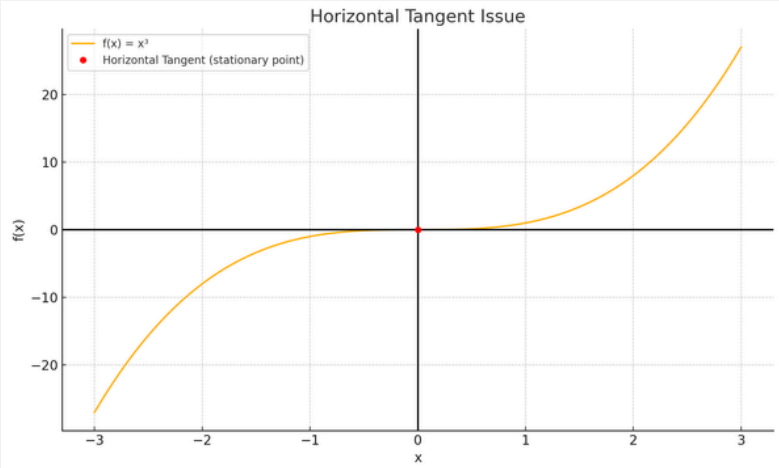

- The blue curve represents the function f(x)=x^2-5.
- The dashed green lines are the tangent lines drawn at each estimate.
- The blue points indicate each iteration point (X(n),F(X(n))).
- The final red point shows the approximate root determined by the method.

## Potential Issues with Newton's Method

- The effectiveness of Newton's method significantly depends on the choice of the initial estimate X(0). Due to its reliance on tangent lines, several problems might occur, including:

## Horizontal Tangent Issue:

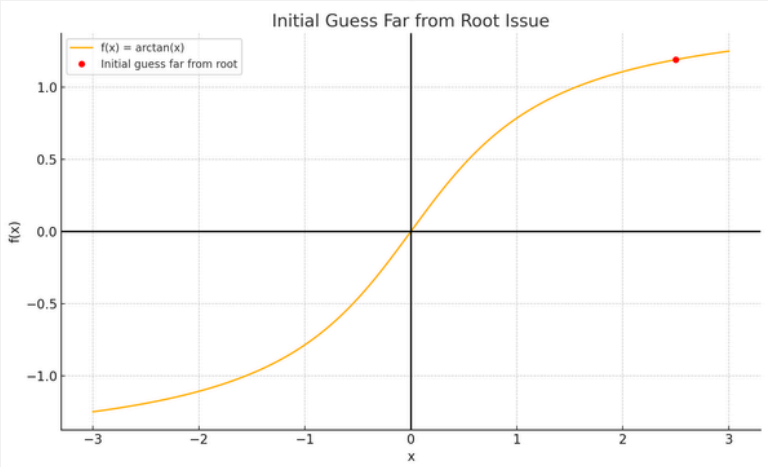
A horizontal tangent occurs if the initial guess X(0) is chosen at a point where the function has a stationary point, i.e., the derivative at this point equals zero f′(X(0)) = 0. In such a scenario, the iterative process cannot proceed, causing the method to fail.



# Introduction to Newton's Method and its Formula

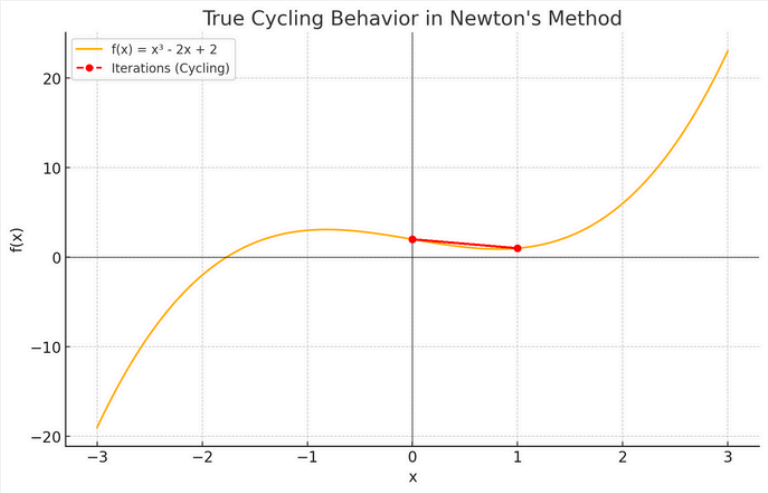
## Initial Guess Far from the Root:

If the initial estimate X(0) is too distant from the actual root, the tangent lines generated might not converge toward the intended solution. Instead, the iterations could diverge or lead to an entirely incorrect root.




## Cycling Behavior:

Newton's method can occasionally become trapped in a repeating cycle, failing to converge to a root. This cycling typically happens around points of inflection, where the curvature of the function changes sign. In such situations, the iteration repeatedly oscillates between two or more points without converging.



# MATLAB Implementation of Newton's Method

Inputs:
- Function f(x).
- Derivative of the function f'(x).
- Initial estimate X(0).

Outputs:
- Approximate root of the function f(x).

Assumptions:
- The derivative f'(x) must not equal zero.
- The function f(x) must be continuous and differentiable.

Procedure Steps:
- Initialize an iterative loop and perform the following steps repeatedly:
  a. Check if |F(X(n))| exceeds a predefined tolerance level.
  b. Compute the new approximation using the formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

  c. Update the iteration count.

**MATLAB Script Input (Newton's Method) 1/2:**

```
function newtonMethod(func, derivFunc, initialGuess)

tolerance = 1e-10;
maxIterations = 100;
iteration = 1;
x_current = initialGuess;

fprintf('\nNewton"s Method Implementation\n');
fprintf('-------------------------------------------------------------\n');
fprintf('Iteration\t    x_n\t\t     f(x_n)\t\t     f"(x_n)\n');
fprintf('-------------------------------------------------------------\n');

while abs(func(x_current)) > tolerance && iteration <= maxIterations

      fprintf('%d\t\t%12.9f\t%12.9f\t%12.9f\n',  iteration,  x_current,  func(x_current),
derivFunc(x_current));

   x_next = x_current - func(x_current)/derivFunc(x_current);
   x_current = x_next;
   iteration = iteration + 1;

end
```

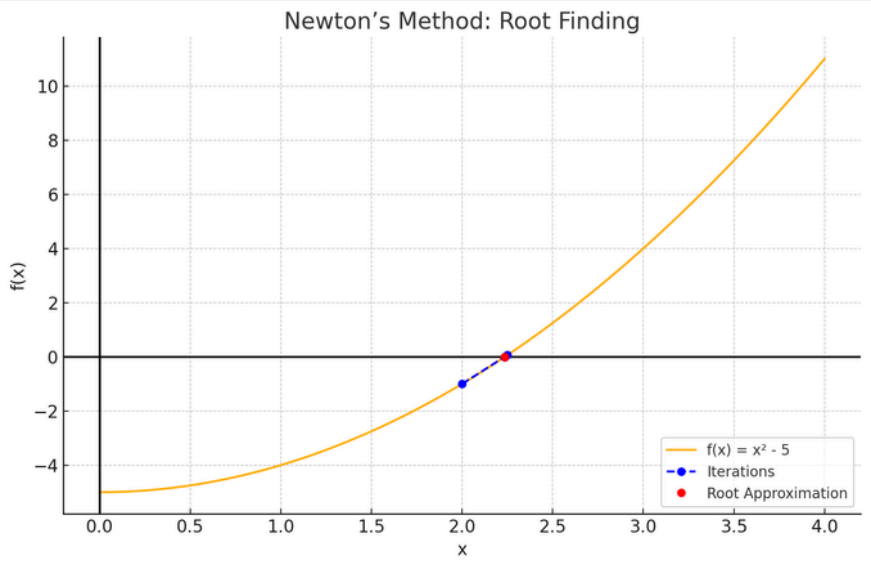
# MATLAB Implementation of Newton's Method

**MATLAB Script Input (Newton's Method) 2/2:**

```
fprintf('-----------------------------------------------------------\n');
if iteration > maxIterations
    fprintf('No roots found within the specified tolerance and iteration limit.\n');
else
    fprintf('Root found: %.9f\n', x_current);
end

fplot(func, [initialGuess-2, initialGuess+2]);
grid on;
hold on;
plot(x_current, func(x_current), 'ro');
xlabel('x');
ylabel('f(x)');
title('Newton's Method: Root Finding');

end
```

**MATLAB Script Output (Newton's Method) :**

| Iteration (n) | x(n) | f(x(n)) | f'(x(n)) |
|---|---|---|---|
| 0 | 2 | -1 | 4 |
| 1 | 2.25 | 0.0625 | 4.5 |
| 2 | 2.23611111111111 | 0.000192901234568054 | 4.47222222222222 |
| 3 | 2.2360679779158 | 0.000000001860473552994 79 | 4.4721359558316 |
| 4 | 2.23606797749979 | 0.000000000000000088817841 | 4.47213595499958 |



# Introduction to Steffensen's Method and it's Formula

Steffensen's Method is a root-finding algorithm in numerical analysis that also provides quadratic convergence, similar to Newton's Method. The key advantage of Steffensen's approach is that it does not require the derivative of the function—making it particularly useful when the derivative is difficult or impossible to compute.

Rather than using the traditional derivative, this method approximates it using forward finite differences, enabling derivative-free convergence while still achieving efficient results. As with Newton's Method, Steffensen's process is iterative, generating a sequence of values x1,x2,x3,..., x(n) that gradually approach the actual root of the function.

$$x_{n+1} = x_n - \frac{[f(x_n)]^2}{f(x_n + f(x_n)) - f(x_n)}$$

This formula assumes the function F(x) is continuous and behaves smoothly near the root. Although the method can sometimes still work with a less accurate initial guess, for best performance, the starting value x0x_0x0 should be close to the true root.

## *Example:*

Consider the function F(x) = x^2-7 Using the initial approximation X(0) = 2.5 Steffensen's Method iterations proceed as follows:

| Iteration (n) | x(n) | f(x(n)) | x(n+1) |
|---|---|---|---|
| 0 | 2.5 | -0.75 | 2.646153846 |
| 1 | 2.646153846 | 0.00109 | 2.645751316 |
| 2 | 2.645751316 | 2.47×10^-8 | 2.645751311 |
| 3 | 2.645751311 | 2.22×10^−16 | 2.645751311 |

The exact root of F(x) = x^2-7 is √7=2.645751311 which Steffensen's Method successfully converges to with high precision.

# Introduction to Steffensen's Method and it's Formula

## Potential Issues with Steffensen's Method

Although Steffensen's Method offers the benefit of derivative-free quadratic convergence, it still shares some of the same weaknesses as Newton's Method—particularly regarding the initial guess.

**Sensitivity to the Initial Guess:**
Just like Newton's Method, the success of Steffensen's approach heavily depends on how close the initial estimate x0x_0x0 is to the actual root. If the starting point is too far, the iterations may:

- Fail to converge,
- Oscillate between values without settling,
- Or even diverge completely from the solution.

**Susceptibility to Cycles:**
Another issue occurs when the function has a point of inflection—a spot where the curvature changes direction. In such cases, the method can enter a cycle, repeatedly bouncing between values without narrowing in on the root.

**Double Function Evaluation Overhead:**
One unique drawback to Steffensen's Method is its computational cost per iteration. For each step, it requires evaluating the function twice:

$$f(x_n) \quad \text{and} \quad f(x_n + f(x_n))$$

This "double hit" can make the method slower than Newton's in practice when working with complex or resource-intensive functions.

# MATLAB Implementation of Steffensen's Method

Inputs:
- Function f(x).
- Initial estimate X(0).

Outputs:
- Approximate root of the function f(x).

Assumptions:
- The function f(x) must be continuous
- Quadratic convergence occurs if the initial guess is close to the actual root

Procedure Steps:
- Initialize an iterative loop and perform the following steps repeatedly:
  a. Check if |F(X(n))| is greater than a specified tolerance.
  b. Calculate the next approximation using the formula

$$x_{n+1} = x_n - \frac{[f(x_n)]^2}{f(x_n + f(x_n)) - f(x_n)}$$

  c. Update the iteration count.
  d. Repeat until the error falls below the tolerance or maximum iterations are reached

**MATLAB Script Input (Steffensen's Method) 1/2:**

```
function steffensenMethod(f, x0)

tolerance = 1e-10;
maxIterations = 100;
iteration = 1;

fprintf('\nSteffensen's Method Implementation\n');
fprintf('-------------------------------------------------------------\n');
fprintf('Iteration\t    x_n\t\t    f(x_n)\t\t    x_{n+1}\n');
fprintf('-------------------------------------------------------------\n');

while abs(f(x0)) > tolerance && iteration <= maxIterations
    fx = f(x0);
    f_fx = f(x0 + fx);
    denominator = f_fx - fx;

    if denominator == 0
        fprintf('Division by zero encountered.\n');
        return;
    end
```

# MATLAB Implementation of Steffensen's Method

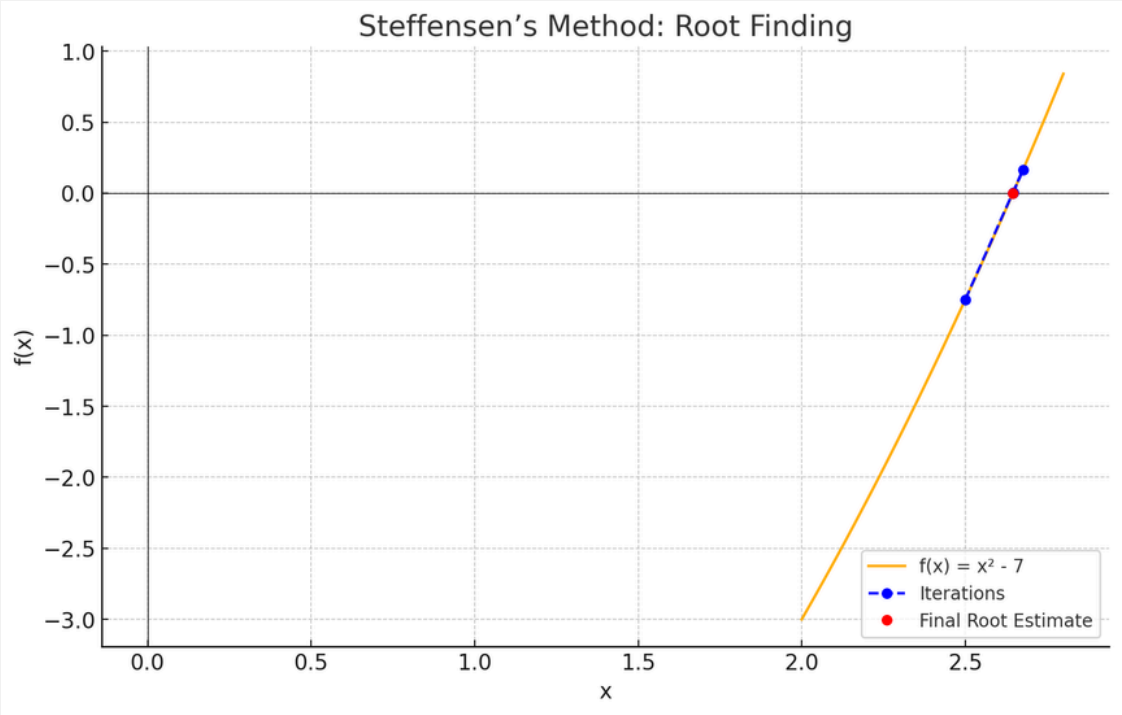
**MATLAB Script Input (Steffensen's Method) 2/2:**

```
x1 = x0 - (fx^2) / denominator;
    fprintf('%d\t\t%12.9f\t%12.9f\t%12.9f\n', iteration, x0, fx, x1);

    if abs(x1 - x0) < tolerance
        break;
    end

    x0 = x1;
    iteration = iteration + 1;
end

fprintf('-------------------------------------------------------------\n');
fprintf('Root found: %.9f\n', x0);

fplot(f, [x0-2, x0+2]);
grid on;
hold on;
plot(x0, f(x0), 'ro');
xlabel('x');
ylabel('f(x)');
title('Steffensen's Method: Root Finding');
hold off;

end
```

# MATLAB Implementation of Steffensen's Method

**MATLAB Script Output (Steffensen's Method) :**

| Iteration (n) | x(n) | f(x(n)) | x(n+1) | Error |
|---|---|---|---|---|
| 0 | 2.5 | -0.75 | 2.676470588 | 0.176470588 |
| 1 | 2.676470588 | 0.163494805 | 2.646832816 | 0.029637772 |
| 2 | 2.646832816 | 0.005723952 | 2.645753097 | 0.001079719 |
| 3 | 2.645753097 | 0.000006563 | 2.645751311 | 0.000001786 |



# Conclusion

- In this project, we explored and implemented two powerful root-finding methods: Newton's Method and Steffensen's Method. Both techniques demonstrated rapid convergence toward the true root when provided with a reasonable initial guess.

- Newton's Method performed efficiently but requires a known derivative, which may not always be available. In contrast, Steffensen's Method does not rely on derivatives and still achieves similar accuracy, though it involves more function evaluations per step.

- On the other hand, Steffensen's Method offers a derivative-free alternative that also achieves quadratic convergence. This makes it especially useful when the derivative is difficult to compute or unavailable. However, it incurs a higher computational cost per iteration due to double function evaluations.

- Our results and graphs confirmed that both methods successfully converged to the correct root. Each method has its advantages, and the best choice depends on whether the derivative is accessible or not.

| **Feature** | **Newton's Method** | **Steffensen's Method** |
|---|---|---|
| **Convergence Rate** | Quadratic | Quadratic |
| **Requires Derivative?** | Yes | No |
| **Iteration Formula** | $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$ | $x_{n+1} = x_n - \frac{[f(x_n)]^2}{f(x_n + f(x_n)) - f(x_n)}$ |
| **Function Calls per Iteration** | 1 function + 1 derivative | 2 function evaluations |
| **Computation Cost** | Lower (if derivative is known) | Slightly higher (Double function |
| **Susceptible to Cycles** | Yes | Yes |
| **Sensitivity to Initial Guess** | High | High |
| **Best Use Case** | Derivative is Available & Simple | Derivative is Unknown \| Complex |